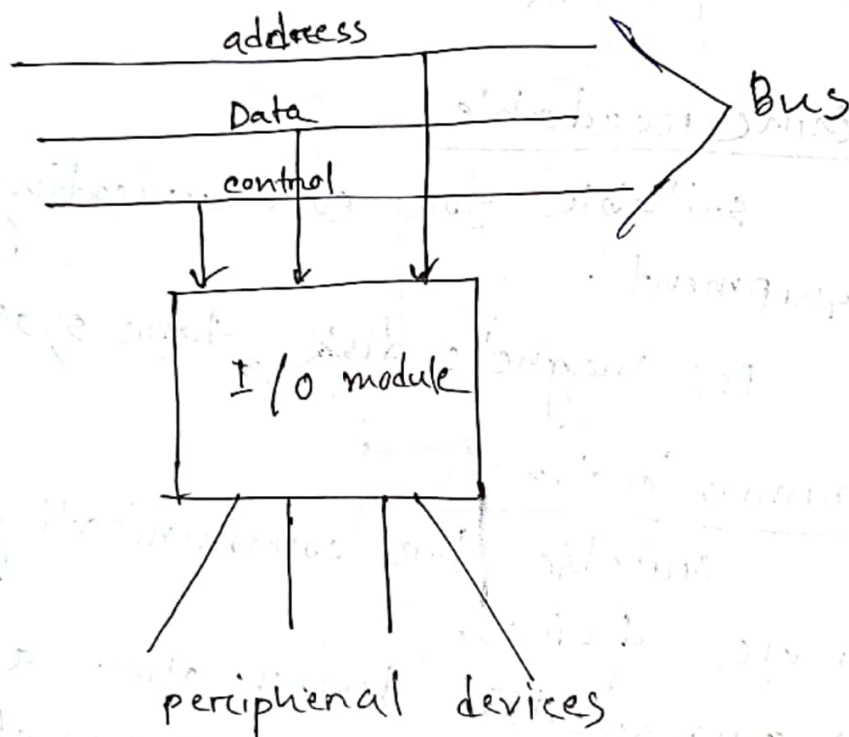


I/O module

bn-7 stu 1

I/O module ?

⇒ Is a device which acts as a bridge between computer system and I/O device.



model of an I/O module.

Ques: External device / peripheral:

An external device connected to an I/O module is often referred to as a peripheral device or simply, a peripheral.

External devices are of three categories:

① Human readable

suitable for communicating with the computer user.

Ex: video display terminals (VDTs), printers.

② Machine readable:

suitable for communicating with equipment.

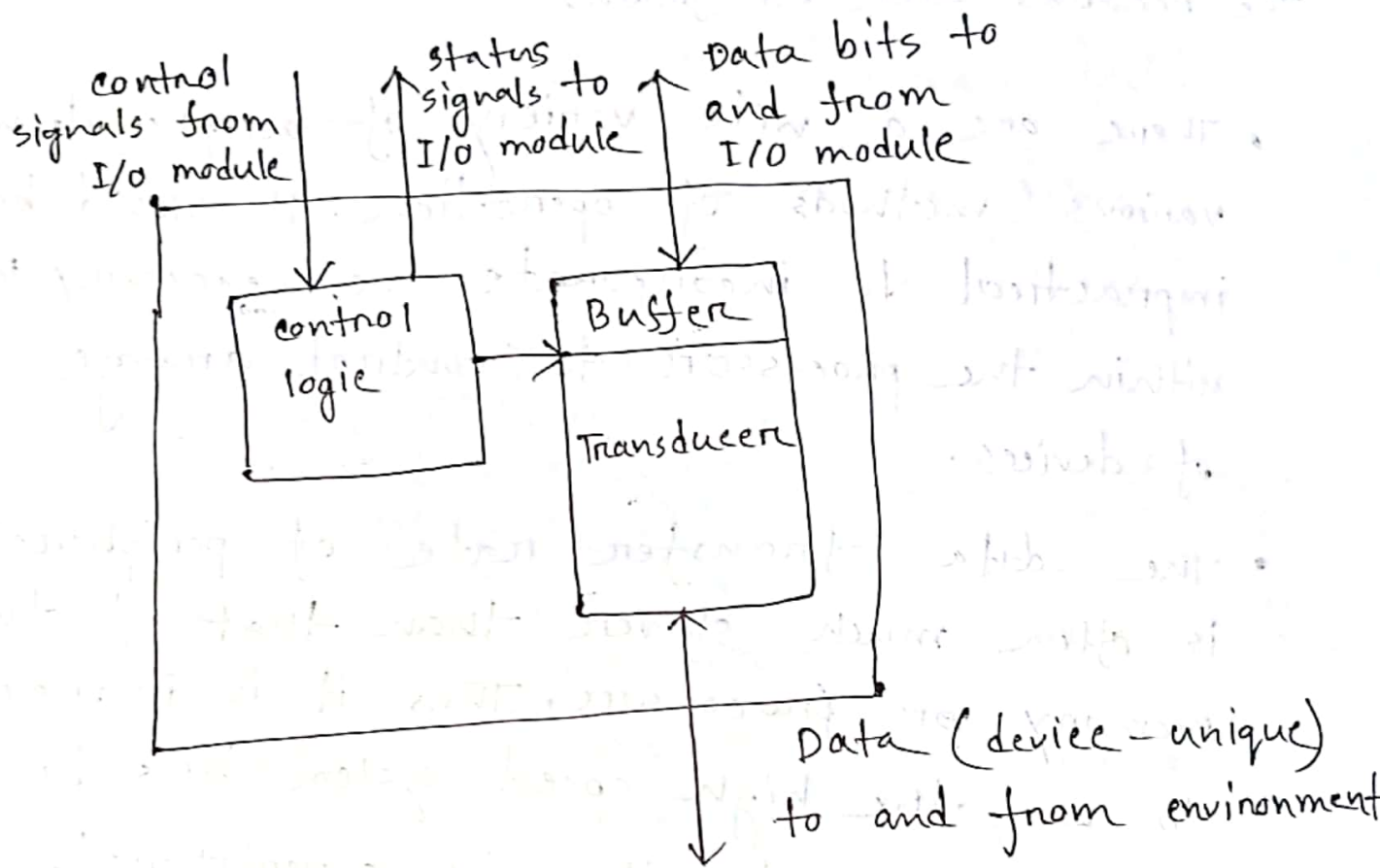
Ex: magnetic disk, tape system.

③ communication:

suitable for communicating with remote devices.

→ communication devices allow a computer to exchange data with → remote devices, another computer.

Block Diagram of an external Device



- IRA = International Reference Alphabet.

P. Question: why input/output module is used in computer system?

The reasons are as follows:

- There are a wide variety of peripherals with various methods of operation. It would be impractical to incorporate the necessary logic within the processor to control, arrange of devices.
- The data transfer rate of peripherals is often much slower than that of the memory or processor. Thus it is impractical to use the high speed system bus to communicate directly with a peripheral.
- on the other hand, the data transfer rate of some peripherals is faster than that of the memory or processor. ~~Thus it is impractical to use the high speed system bus to communicate directly with a peripheral.~~ again, the mismatched would lead to inefficiencies if not managed properly.

- peripherals often use different data formats and word lengths than the computer to which they are attached.

Thus, an I/O module is required. ~~this module has two major functions~~

* Disk Drive:

contains electronics for exchanging data, control and status signals with an I/O module plus the electronics for controlling the disk read/write mechanism.

I/O module function:

P. Question.1: Describe the function of I/O module.

⇒ The major functions or requirements for an I/O module fall into the following categories:

❑ control and timing:

I/O module includes control and timing to coordinate the flow of traffic between internal resources and external devices. The control of the transfer of data from external devices to processor following ~~steps~~ ~~some~~ the ~~steps~~ status of I/O module.

❑ Processor communication:

The I/O module must communicate with the processor and with the external device. Processor communication involves the following -

- command decoding
- Data
- status reporting
- Address recognition.

☐ Device communication:

I/O module must be able to perform device communication. By this communication, I/O module ensure the status of external devices. it involves

- commands.
- status information.
- Data.

☐ Data buffering:

An essential task of an I/O module is data buffering. Data coming from main memory are sent to an I/O module in a rapid burst. The data are buffered in the I/O module and then sent to the peripheral device at its data rate.

❑ Error detection:

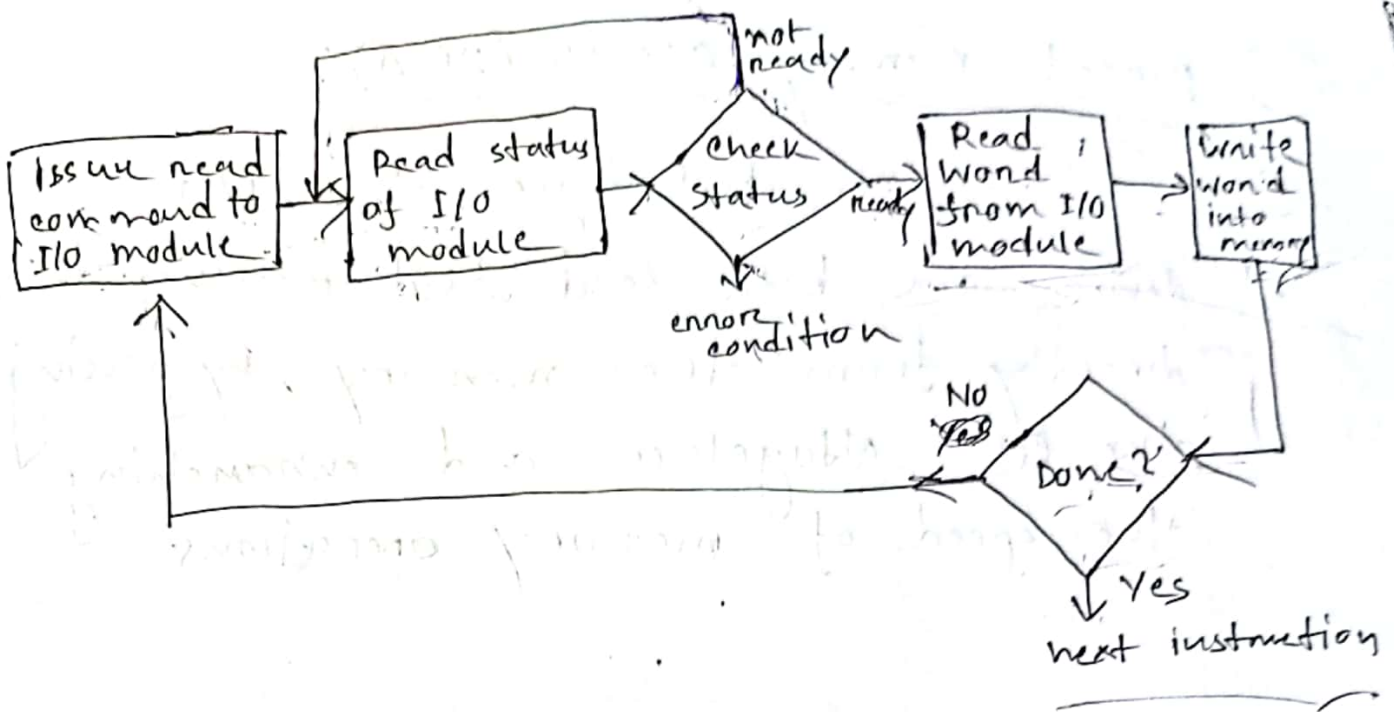
An I/O module is often responsible for error detection and for subsequently reporting errors to the processor. One class of errors includes mechanical and electrical malfunctions reported by the device.

P. Question: List and briefly define three techniques for performing I/O.

⇒ Three techniques are possible for I/O operations they are

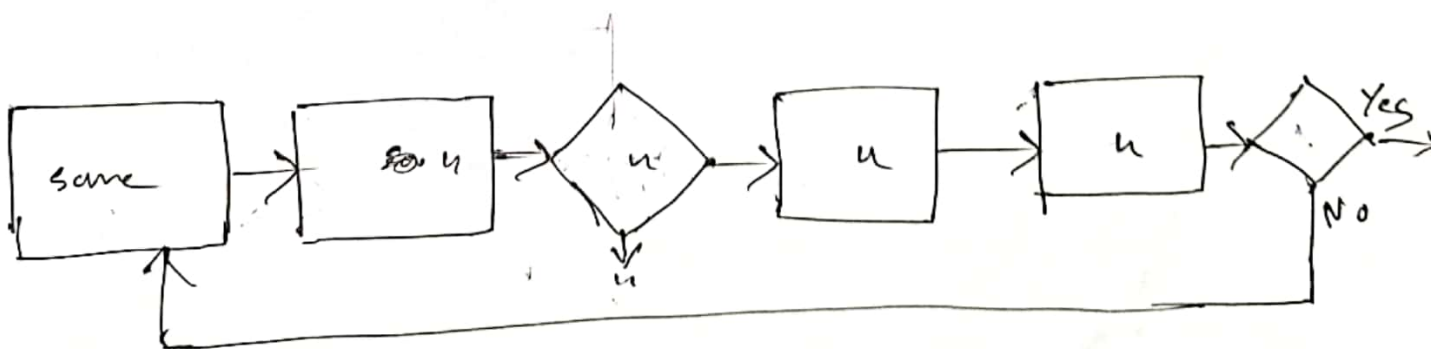
(i) programmed I/O:

In this case, the transfer of data from an I/O device to the memory will require the CPU to initiate a program. The CPU remains in a loop until the I/O device is prepared to make the data transfer.



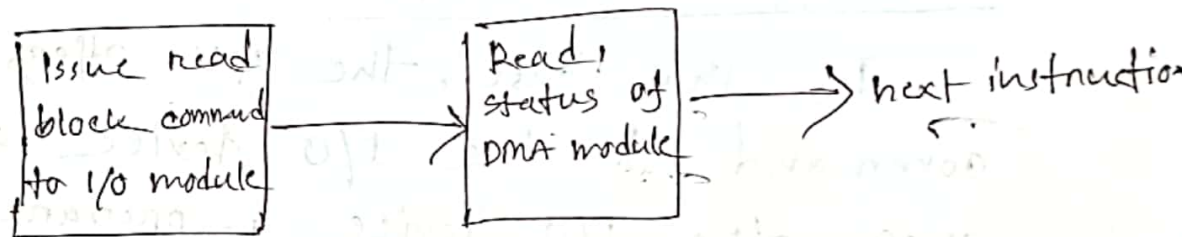
② Interrupt driven I/O:

In this case, the CPU offers a read command to the I/O device, and once the I/O device is prepared to transfer data, it sends an interrupt signal to the processor, then the CPU checks the status of the device and then action according to the memory.



③ Direct memory access (DMA):

In this technique, the I/O device can both send and receive data directly from the memory, by passing the CPU altogether and enhancing the speed of memory operations.



P. Questions mention steps of interrupt driven I/O.

⇒ Interrupt driven I/O is an alternative scheme dealing with I/O. Interrupt I/O is a way of controlling input/output activity where by a peripheral or terminal that needs to make or receive a data transfer sends a signal.

The processing steps are following: -

- ① A device driver initiates an I/O request on behalf of a process.
- ② The device driver signals the I/O controller for the proper device.
- ③ The device signals the I/O controller that is ready to retrieve input, the output is complete or that an error has been generated.
- ④ The CPU receives the interrupt signal on the interrupt request line and transfers control over the interrupt handler routine.

⑤. The interrupt handler determines the cause of the interrupt, performs the necessary processing and executes a "return from" interrupt instruction.

⑥ The CPU returns to the execution state prior to the interrupt being signaled.

7. The CPU continues processing until the cycle begins again.

" Interrupt drive I/O diagram

माकल रात mark अनुमायी "

Design Issues:

There are two main problems that arise for interrupt I/O, which are: -

- There are multiple I/O modules, how should the processor determine the device that issued the interrupt signal?
- How does the processor decide which module to process when multiple interrupts have occurred?

Now, there are four main ways to counter these problems, which are: -

- Multiple interrupt lines.
- Software poll.
- Daisy chain (Hardware poll, vectored)
- Bus arbitration. (vectored)

pipelining

Processor organization:

the things must do .

- Fetch instruction .
- Interpret instruction .
- Fetch data .
- process data
- write data .

Register organization:

- . Register in CPU performs two roles .
 - User - visible registers .
 - control and status registers .

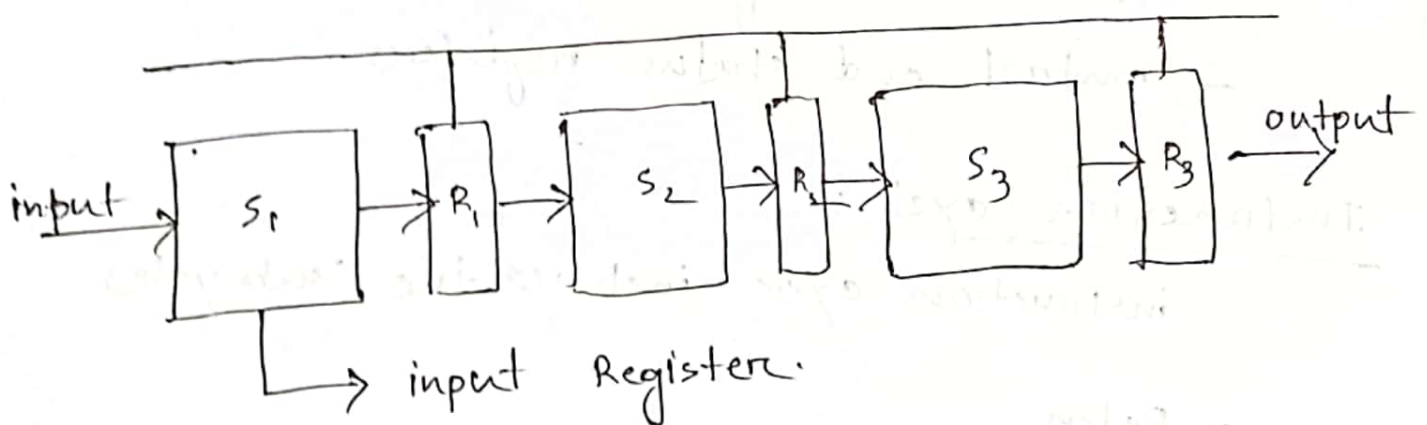
Instruction cycle:

instruction cycle includes the subcycles

- Fetch .
- Execute .
- Interrupt .

pipelining:

- pipelining is the process of accumulating instruction from the processor through a pipeline.
- Multiple instructions are overlapped during execution.
- pipeline is divided into stages, which are connected with one another.
- Instructions enter from one end and exit from another end.
- pipelining increases the overall instruction throughput.



- 2 categories
- 1. Arithmetic.
 - 2. Instruction.

Instruction pipeline:

• In this stream of instruction can be executed by, ~~overlapping fetch~~ -

- overlapping fetch

- decode

- execute phases of an instruction cycle.

• Increases throughput.

→ conflicts,

1. Timing variations:

It have - Different instructions having

↳ Different operand requirements

↳ Different processing time.

2. Data Hazards :-

- Different executions referring to some data.

3. Interrupts :- Interrupts set unwanted instruction into the instruction stream.

1. Branching:

- If the present instruction is conditional branch, it will lead us to the next instruction, then the next instruction may not be known until the current one is processed.

5. Data dependency:

• when an instruction depends upon the result of a previous instruction but the result is yet unavailable.

→ Advantages

↳ cycle time of processor is reduced.

↳ Increase the throughput of the system.

↳ makes the system reliable.

Disadvantages:

↳ The design of pipelined processor is complex and costly to manufacture.

↳ The instruction latency is more.

Question: chapter 12

Q) what roles are performed by cpu registers?

⇒ In general sense, cpu registers are small storage units within the cpu that are used to hold data that is being processed by the cpu. The specific roles that registers play can vary depending on the specific architecture of the cpu. but some common tasks that registers are used for includes:—

1. Holding data that is being processed or operated by the cpu.

2. Storing intermediate results of calculations performed by the cpu.

3. Holding addresses of memory locations that the cpu is currently accessing or needs to access.

4. Holding status information, such as

- current state of the processor.

- result of arithmetic operations.

5. Storing control information, such as the current instruction being executed on the location of the next instruction to be executed.

- overall purpose of cpu registers:—

- provide the cpu with quick access to data and instructions that it needs to execute in order to perform its tasks.

Q. What categories of data are commonly supported by visible registers?

- visible registers are registers that are directly accessible to programs running on the CPU.

Hence are some examples of the types of data that are commonly supported by visible registers.

1. Integer values: whole numbers with no fractional component.

↳ signed numbers.

↳ unsigned numbers.

2. Floating point values.

3. pointers: special values that represent the address of a location in memory.

4. status and control information:

↳ current status of the processor -

↳ results of arithmetic operations.

These registers are typically not directly accessible to programs, but are used by the CPU to control its own operations.

② What is the function of condition codes?

— conditional codes stored in "flag registers" are known as flags or status flags, are bits within the CPU that are used to store →

- Information about the current state of processor.

- Result of arithmetic operations.

→ Flags supported by CPU.

1. Carry flag :-

- set when an arithmetic operation generates a carry-out or borrow-in.
- used to detect, overflow / underflow conditions in arithmetic operations.

2. Zero flag :- set when the result of an arithmetic / logic operation is zero.

3. Sign flag : set when result of an operation is too large to be represented in the destination register.

- Detects overflow / underflow.

→ Flag are used with conditional jump instructions.

- to allow the program to branch to different parts of the code based on the values of the flag.

• Implements decision-making logics in program.

Q What is a program status word?

→ PSW is a special register in computer's CPU that stores the current state of a program.

contains information about:

- The current instruction being executed.
- status of the program (running or paused?)
- Any error or expectation conditions that have occurred.

→ controls the flow of the program.

→ handles interrupts and exceptions.

⇒ referred as "condition code register" / "processor status" register.

12.1: (a) If the last operation performed on a computer with an 8-bit word was an addition in which the two operands were 2 and 3. what would be the value of the following flags?

- carry.
- zero.
- overflow.
- sign.
- Even parity.
- Half carry.

Answer:

$$\begin{array}{r} 2 \rightarrow 00000010 \\ 3 \rightarrow 00000011 \\ \hline 00000101 \end{array}$$

From the result we can say,

- carry flag will be zero as there is no carry
- zero flag - 0

• overflow flag - 0

• sign flag - 0

• Even parity flag - 0

• Half-carry flag - 1

• because the addition of the low-order bits (which are 2 and 3) generates a carry-out.

12.2.1:

		1	2	3	4	5
instruction 1		F	E			
" 2			F	E		
" 3				F	E	
" 4					F	E

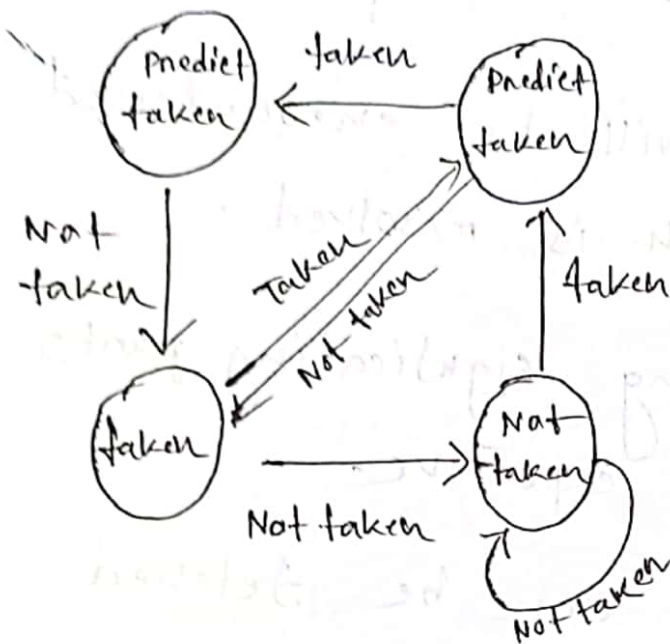
F = Fetch

E = Execution

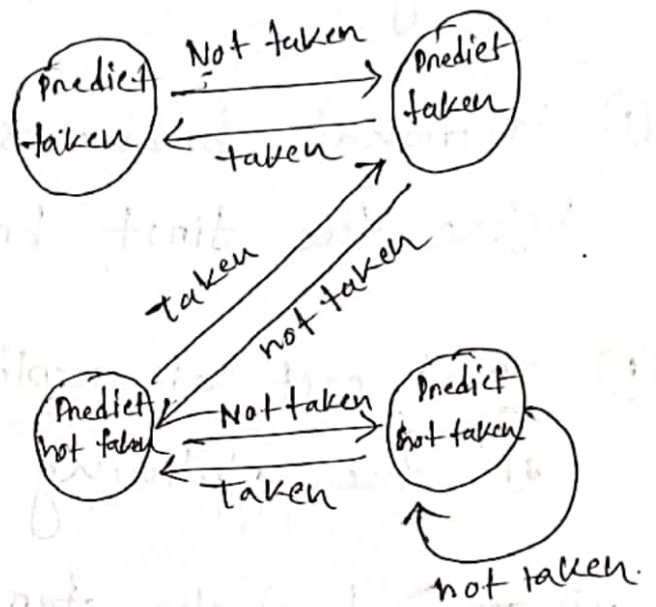
12.4 Limitation of the multiple-stream approach to dealing with branches in a pipeline.

- (i) Additional branches will be encountered before the first branch is resolved.
- (ii) The cost of replicating significant parts of the pipelining is expensive.
- (iii) The branch target can be fetched until its address is determined. This causes a delay in loading one of the streams.

12.4. consider the state Diagram.



[A]



[B]

call the first state diagram strategy A.
 strategy A corresponds to the following behaviour:-

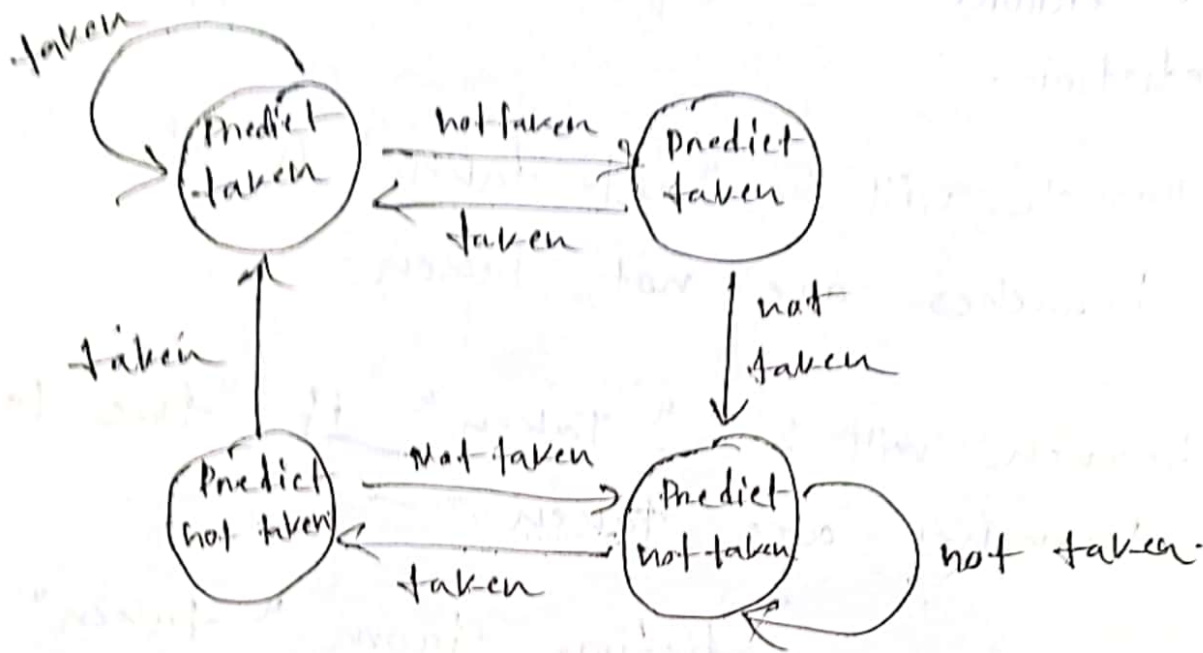
- if both of the last two branches of the given instruction have "not taken" the branch, then predict that the branch will not be taken.

otherwise, predict that the branch will be taken.

for strategy B $\rightarrow$

- Two errors are required to change the prediction.
- A Branch will be "not taken" if the last two branches are not taken.
- A branch will be "Taken" if the last two branches are taken.
- To change a prediction from "taken" to "not taken". Two "not taken" branches are required and vice-versa.
- However, if there is a change in prediction followed by an error, the previous prediction is restored.

b) comparison of strategy "A" and "B" and "C"



c)

- strategy A works best when it is usually the case that branches taken.
- In strategy A and C
Two wrong guesses are required to change the prediction.
so, when, most branches are part of a loop. strategy B and C are superior than strategy A.

Q why is a two stage instruction cycle pipeline unlikely to cut the instruction cycle time in half compared with the use of no pipeline.

⇒ two stage instruction pipeline can help to improve the instruction cycle time its half.

Because of several factors that can limit that performances,

Factor's are,

Time required to fetch and decode the instruction.

→ Time will be maximum, if the instruction are located in slow memory.

- complex, instruction
- Instruction require lot of decoding.

- Time required to execute the instruction
- Time required to move data between stages.
- Time required to handle pipeline stalls and flushes.

→ overall for these factors mentioned above, two stage instruction pipeline can help to improve the instruction cycle, but cannot cut the time in half.

② list and briefly explain various ways in which an instruction pipeline can deal with conditional branch instruction?

⇒ There are several ways that ~~can~~ an instruction pipeline can deal with conditional branch instruction.

① Delayed Branching:

- This technique delays the execution of the branch instructions until the next instruction has been fetched.

- This allows the pipeline to continue without stalling.

② Branch prediction:

- A branch prediction unit is used to predict the outcome of a branch.

- If prediction is correct, the pipeline can continue without stalling.

If it is incorrect,

- pipeline must be flushed and correct instruction fetched.

③ Branch target buffering:

- BTB is used to store the targets of recently taken branches.

④ Hybrid Approach:

- Some system use combination of all these above mentioned technique to handle conditional branch instruction in the pipeline.

How are history bits used for branch prediction?

→ On computer architecture, Branch prediction is used to try to guess the outcome of a conditional branch instruction before the instruction is actually executed.

No need { the idea is to avoid the pipeline to stall / stop. This would occur if the processor had to wait until the instruction determine which way the branch should go.

→ ~~then~~ History bit is a single bit memory that is used to store the outcomes of a previous branch.

For example,

If the processor has just executed a branch instruction that was taken (meaning the branch was - followed)

The history bit for the branch would set 1 or 0 for vice versa.

When the processor encounters a new branch instruction, it can use the history bit for that branch to try to predict the outcome of the branch. If the history bit is 1, the branch will be taken again and start fetching instruction from the target address.

If it is 0, branch will not be taken and continue to fetch instruction from new address.

Chapter - 10

Q what are the typical elements of a machine instruction?

⇒ Essential elements of a computer instruction

- op code: specifies the operation to be performed

- source and destination operand references:

specifies the input output locations for the operation.

- Next instruction reference:

- is usually implicit.

→ Registers and memory hold source and destination operands.

Q If an instruction contains four addresses, what might be the purpose of each address?

↳ Two operands.

- contains the values needed for the instruction.

↳ one result:

- contains the values needed for the instruction.

↳ Address of the next instruction.

- Register holds the next instruction that needs to be performed.

② List and briefly explain five important instruction set design issues.

- operation repertoire

- How many and which operations to provide?

- How complex operations should be?

- Data type

- The various types of data upon which operations should be performed.

Instruction format:

- Instruction length (in bits)
- Number of addresses.
- size of various fields.

Registers:

- Number of cpu registers that can be referenced by instruction.
- Their use.

Addressing:

- The mode or modes by which the address of an operand is specified.

Q) what types of operands are typical in machine instruction set?

- Addresses.

- Numbers.

- characters.

- logical data.

⑥ What is the relationship between the IRA character code and packed decimal representation?

• International Reference Alphabet (IRA)

- encoded characters are almost always stored and transmitted using 8 bits per character.

→ IRA bit pattern: 011XXXXX.

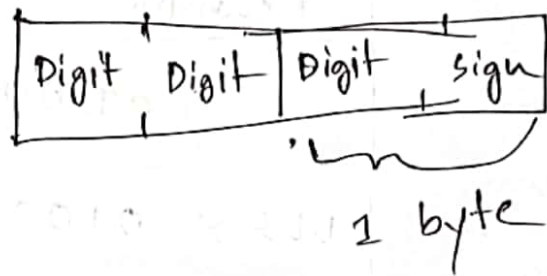
↳ Digits 0-9 are represented by their binary equivalents. 0000 through 1001, in the right most 4 bits.

- which is same code as packed decimal.

• packed decimal format:

→ This format means that each byte of storage (except the low order byte) can contain two decimal numbers.

- The low order byte contains one digit in the left most portion and the sign (positive or negative) in the Right most portion.



Q What is the difference between an arithmetic shift and a logical shift?

Arithmetic shift

→ An arithmetic shift is a type of bit shift that preserves the sign of a number being shifted.

→ In arithmetic left shift,
• zeros are inserted at the LSB (right) and MSB (left) bits are discarded

Logical shift

→ Doesn't preserve the sign of the number being shifted.

→ In a logical left shift, zeros are inserted at the LSB and MSB are discarded.

AS

→ In arithmetic Right shift,

- msb bits are preserved
- lsb bits are discarded

Example:

10100111

ALS → 01001110

ARS → 11010011

LS

→ In a logical Right shift

- msb are discarded
- lsb bits are filled with 0's

Example:

10100111

LLS → 01001110

LRS → 01010011

Q. why are transfer of control instructions needed?

- Transfer of control instructions: also known as branching or jump instructions, allow the program to make decisions and to execute different parts of code depending on the current state of the program on the inputs of a program.

without transfer of control program, it would be difficult to implement decision making logic or to create loops that repeat a section of code multiple times.

→ types of transfer control instruction →

1. conditional branches: Allow the program to branch to a different part of the code based on value of a particular condition.
2. unconditional branches: jumps to a different part of the code without regard to any particular condition.
3. subroutine calls: Allow the program to execute a subroutine overall. transfer of control instruction are an important part of any programming language and essential for implementing complex logic and control structures program.

Q List and briefly explain two common ways of generating the condition to be tested in a conditional branch instruction.

⇒ There are many different ways of generating the condition to be tested in a conditional branch instruction.

Hence are two common ways -

1. using flag bits of Flag registers.
2. using three-address instruction format

↳ used to perform a comparison and specify a branch in the same instruction.

Q Nesting of procedures.

↳ Being able to call a procedure from within a procedure.

Q three possible places for storing the return address for a procedure return.

1. Register
2. start of called procedure
3. The top of the stack.

Q what is a reentrant procedure?

• It is a procedure in which it is possible to call open to it at the same time.

Example: recursive function.

Q what is reverse polish notation?

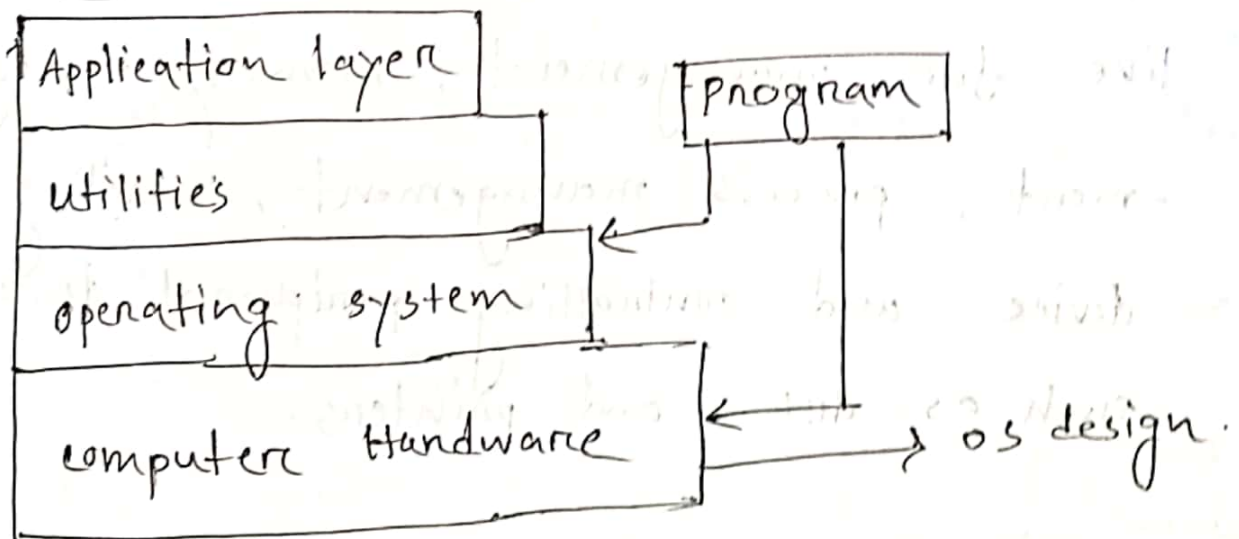
- Notation in which the

operating system (ch-08)

operating system services:-

1. program creation.
2. program execution.
3. Access to I/O device.
4. controll access to file.
5. system access.
6. Error detection and response.
7. Accounting.

End
user



Types of OS:

- (i) interactive
- (ii) uniprogram / single program.
- (iii) Batch.
- (iv) Multi program.

Q.1 what is an operating system?

⇒ An operating system is an interface between computer user and computer hardware. An operating system is a software which perform all basic task like file management, memory management, process management, handling I/O device and controlling peripheral devices such as disk and printers.

8.2 Define services provided by as:

1. Program execution:- OS manages how a program is going to be executed. It loads the program into the memory after executed.
2. Access to I/O device:- OS manages the I/O devices and establish communication between user and device drivers.
3. Control to access file:- OS helps in managing files also.
4. Error detection and response:- OS handles error in CPU and in I/O device.
5. Resource management:- system resource are shared between various process. It is the OS that manages the resource sharing.

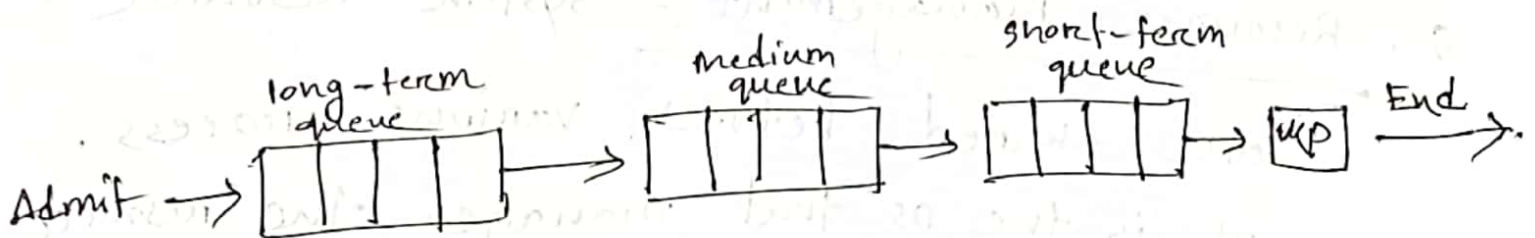
8.3 List major types of OS scheduling:-

There are mainly three types of OS scheduling.

1. Long term scheduling:- The decision to add to the pool of process to be executed.

2. Medium Term scheduling:- The decision to add to the number of process that are partially or fully in main memory.

3. Short term scheduling:- The decision to add to which available process will be executed by the processor.



8.4 Difference between process and program are:

program

① Program is a set of instructions designed to complete a specific task.

② Program is a secondary entity as it resides in the secondary memory.

③ Program does not have any control block.

process

① process is an instance of an execution program.

② process is an active entity as it is created during execution and loaded into the main memory.

③ process has its own control block called process control block.

Chapter: 5

5.1] what are the key properties of semiconductor memory?

⇒ there are several key properties of semiconductor memory -

1. Non-volatile: semiconductor memory ~~is~~ stores data even when power is turned off.
2. Random access: individual memory location can be accessed directly rather than read the entire memory.
3. Read/write: semiconductor memory allows both read and write of data to the same memory location.
4. Durability: semiconductor memory has a long life span.
5. low cost: semiconductor memory are comparatively low cost.

6. High density :- semiconductor memory can store large amounts of data in a small space.

7. Speed :- semiconductor memory are generally faster than other types of memory.

5.2 :- what are two sense in which the term random accessed memory is used?

⇒ In the first sense random access memory (RAM) is a type of computer memory that can be accessed randomly.

meaning that, any memory location can be accessed without having to go through the previous location.

This is in contrast to sequential access memory, which can only be accessed in a fixed sequence.

In second sense, RAM is used more badly to refer to any form of computer memory that can be accessed quickly meaning that it can be read and write at high speeds.

This includes various types of memory such as dynamic RAM (DRAM), static RAM (SRAM) and cache memory.

Q.3) What is the difference between DRAM and SRAM in terms of application.

DRAM

SRAM

- | | |
|---|--|
| ① DRAM stores for dynamic Random access memory. | ① SRAM stands for static Random access memory. |
| ② DRAM stores data with the help of capacitors. | ② SRAM store information with the help of transistors. |

D

③ In ~~stores~~ DRAM contents of capacitors need to be refreshed periodically.

④ D-RAM provide slower speed of data read write.

⑤ DRAM consume less power

⑥ D-RAM are used as main memory in computer system.

S

③ In SRAM contents need to be refreshed.

④ SRAM provide faster speed of data read write.

⑤ S-RAM consumer more power.

⑥ SRAM are used as cache memory in computer and other computing devices.

5.4 Difference between DRAM and SRAM in terms of characteristics such as speed, size and cost.

characteristic	S-RAM	D-RAM
1. speed	faster	slower
2. cost	expensive	cheap
3. size	small	large
4. used in	cache memory	main memory
5. power consumption	High	Low

(5.5) Explain why one type of RAM is considered to be analog and the other digital.

⇒ There are two main types of RAM.

Dynamic RAM (D-RAM) and static RAM.

Dynamic Ram stores data as a charge in a capacitor. while static RAM stores data in a flip flop circuit. Because the

charge in a capacitor can vary and take on an infinite number of

values. D-RAM is considered to be an analog technology.

5.1 What are the key properties of semiconductor memory?

⇒ There are several key properties of semiconductor memory:-

1. Non volatile :- Semiconductor memory stores data even when power is turned off.
2. Random access :- Individual memory location can be accessed directly rather than read the entire memory.
3. Read/Write :- Semiconductor memory allows both read and write of data to the same memory location.
4. Durability :- Semiconductor memory has a long life span.
5. Low cost :- Semiconductor memory are comparatively low cost.
6. High Density :- Semiconductor memory can store large amounts of data in a small space.
7. Speed :- Semiconductor memory are generally faster than other types of memory.

2.2] What are two sense in which the term random access memory is used?

→ On the first sense random access memory (RAM) refers to a type of computer memory that can be accessed randomly, meaning that, any memory location can be accessed without having to go through the previous location. This is in contrast to sequential access memory, which can only be accessed in a fixed sequence.

On second sense, RAM is used more badly to refer to any form of computer memory that can be accessed quickly, meaning that it can be read and write at high speeds.

This includes various types of memory such as dynamic RAM (DRAM), static RAM (SRAM) and cache memory.

5.3] What is the difference between DRAM and SRAM in terms of application

DRAM

SRAM

- | | |
|--|---|
| 1. DRAM stands for Dynamic Random access memory. | 1. SRAM stands for Static Random access memory. |
| 2. DRAM stores data with the help of capacitors. | 2. SRAM stores information with the help of transistors. |
| 3. In DRAM contents of capacitors need to be refreshed periodically. | 3. In SRAM contents need not to be refreshed. |
| 4. DRAM provide slower speed of data read write. | 4. SRAM provide faster speed of data read write. |
| 5. DRAM consume less power. | 5. SRAM consumes more power. |
| 6. DRAM are used as main memory in computer system. | 6. SRAM are used as cache memory in computer and other computing devices. |

5.4

Difference between DRAM and SRAM in terms of characteristic such as speed, size and cost.

Characteristic	SRAM	DRAM
Speed	SRAM provide faster speed of data read and write.	DRAM provide slower speed of data read and write.
Cost	SRAM is expensive.	DRAM is cheap
Size	SRAM is small in size.	DRAM is large
used in	used in cache memory	used as main memory
power consumption	High	Low

5.5] Explain why one type of RAM is considered to be analog and the other Digital.

⇒ There are two main types of RAM. Dynamic RAM (DRAM) and static RAM (SRAM). Dynamic RAM stores data as a charge in a capacitor, while static RAM stores data in a flip flop circuit. Because the charge in a capacitor can vary and take on an infinite number of values, DRAM is considered to be an analog technology.

On contrast the flip-flop circuit used in SRAM can only store two states (0 or 1), so it is considered to be a digital technology.

5.6 What are some applications of ROM?

⇒ ROM stands for read-only-memory. It refers to types of memory that are permanently written and cannot be easily rewritten or modified by a user or CPU. There are several common applications for ROM:-

1. BIOS:- The BIOS of a computer is stored in ROM and contains the instructions for booting the computer.

2. Firmware:- Firmware is stored in ROM.

3. Game Console:- Game console uses ROM to store the games that are played on them. ROM contains the instructions and data needed to run the game.

4. Embedded system:- ROM is often used in embedded systems.

5. Appliance:- Many household appliances, such as microwave ovens and washing machines, use ROM to store the instructions for their functions.

5.7 Difference between EPROM, EEPROM and Flash memory?

EPROM	EEPROM	Flash memory
① Erasable programmable Read only memory	① Electrically erasable programmable read only memory.	① Intermediate between EPROM and EEPROM.
② Erase process can be performed repeatedly.	② Can be rewritten at any time without erasing prior content.	② Uses electrical erasing technology.
③ Based on NOR gate.	③ Based on NOR gate.	③ Based on NAND gate.
④ More expensive than PROM	⑤ More expensive than EPROM	⑤ Less expensive.

6.9 What is parity bit?

A parity bit is a bit that is added to a group of data bits to ensure that the number of 1's in the group is even or odd. It is used as a simple form of error detection in digital communication. The idea is that, if any bits in the group are changed from 1 to 0 or vice versa, due to transmission error, the parity bit will detect the error.

For example consider a group of 4 data bits 1010. If we want to use even parity we could add a parity bit such that

the number of 1's in the group is even. In this case the parity bit would be a 0, because there are already 2 1's in the group.

1 1 1 0 $\rightarrow$ add parity

0 1 1 0 $\rightarrow$ even parity.

5.10 How is the syndrome for the Hamming code interpreted?

The syndrome for hamming code is a set of bits that can be used to identify and correct errors that have been occurred in a transmitted message.

if the syndrome is all zeros, it indicates that there are no errors in the message. If the syndrome contains one bit set to 1, then an error has occurred, but the error is negligible.

if the syndrome contains more than one bit set to 1, then a position error has occurred. And the error can be corrected by flipping the bit position.

Addressing Mode

Chapter 11

Q(12.1-12.7) :- What is addressing? Discuss different types of addressing modes →

⇒ Addressing refers to the way that the processor or other hardware components in a computer access memory or other resources.

Listing different addressing mode :-

1. Immediate addressing mode :-

⊗ on this mode the operand is specified in the instruction itself and easily identified.

Algo = Operand A

Ex: MOV CL, 12H

Advantage :-

⊗ No memory reference

Disadvantage :-

⊗ Limited operand magnitude.

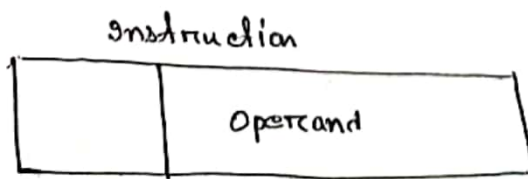
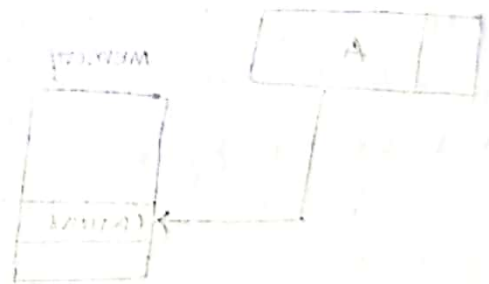


Fig: Immediate

2. Direct Addressing :-

On this mode, address of the operand is directly specified in the instruction.

Algo : $EA = A$

Ex :- `MOV CL, [2000H]`

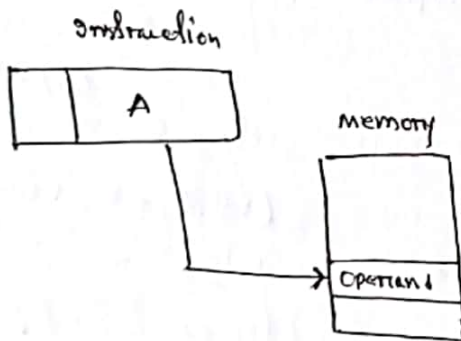
Advantage :-

⊗ Simple

Disadvantage :-

⊗ Limited address space

Fig :-



3. Indirect Addressing :-

On indirect addressing mode, address is given by a register.

Algo : $EA = (A)$

Advantage

Ex : ⊗

MOV CL, [BX] (register indirect addressing)

MOV CL, [BX+0.3H] (Register Relative addressing mode)

MOV CL, [BX+SI] (Base Index Addressing Mode)

MOV CL, [BX+SI+0.3H] (Base relative plus index address mode)

Advantage :-

① Large address space.

Disadvantage

① multiple memory reference.

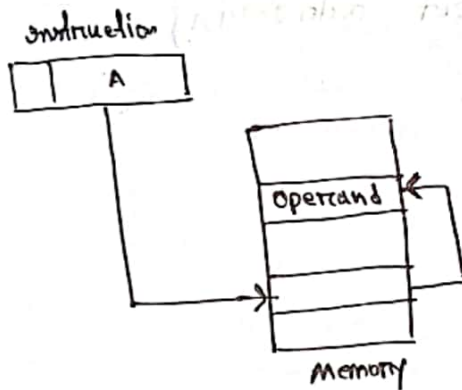


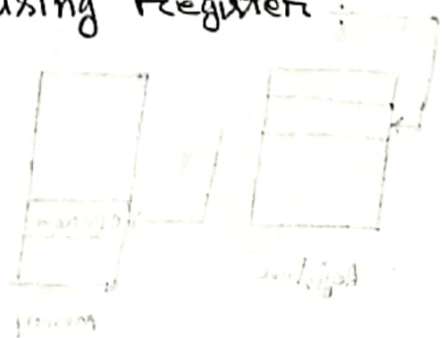
Fig: Indirect addressing

4. Register addressing mode :-

on this mode operand are specified using register.

Algo :- $EA = R$

Exam :-
MOV CL, DL
MOV AX, BX

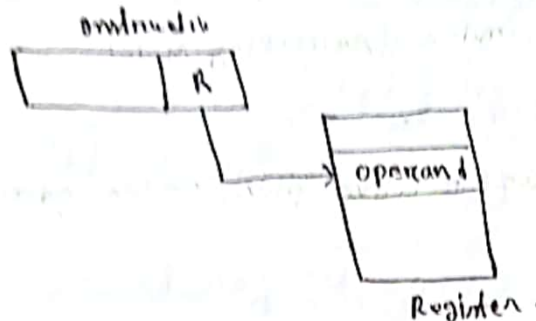


Advantages :-

① No memory reference

Disadvantages :-

① Limited address space.



5. Register Indirect Addressing :-

This is the most basic form of indirect addressing. Address is given by register.

Algo: $EA(R)$

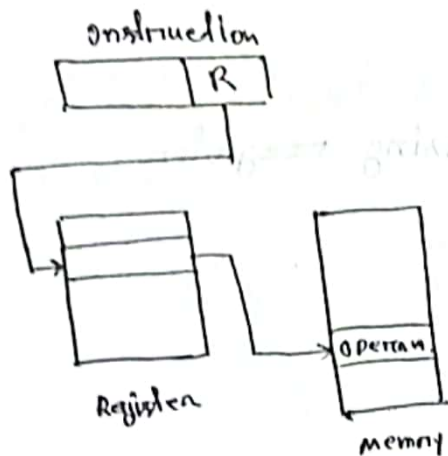
Ex :- `MOV AL, [BX]`

Advantage :-

① Large address space.

Disadvantages :-

① Extra memory ref



6. Displacement addressing :-

Algo: $EA = A + (R)$

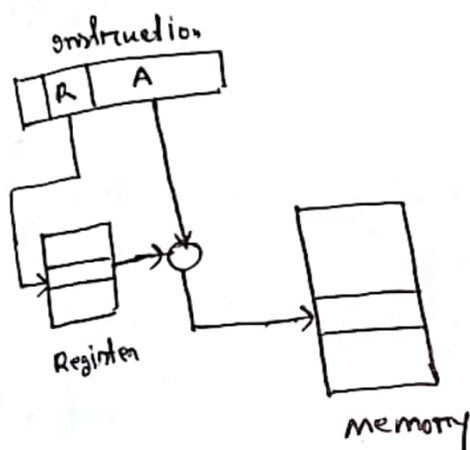
Example :- `MOV EAX, [BX + 0.3H]`

Advantages :-

① Flexibility

Disadvantages :

① complexity



7. STACK addressing :- (IMPLIED Addressing) :-

on this mode the operand is implied in the instruction.

Algo: $EA = \text{TOP of stack}$

Ex: `STC` (set the carry flag in the Flag register)

`CMC` (complement the carry Flag in the Flag register)

Advantages

① Instructions are generally one byte.

Disadvantages

① Limited applicability.



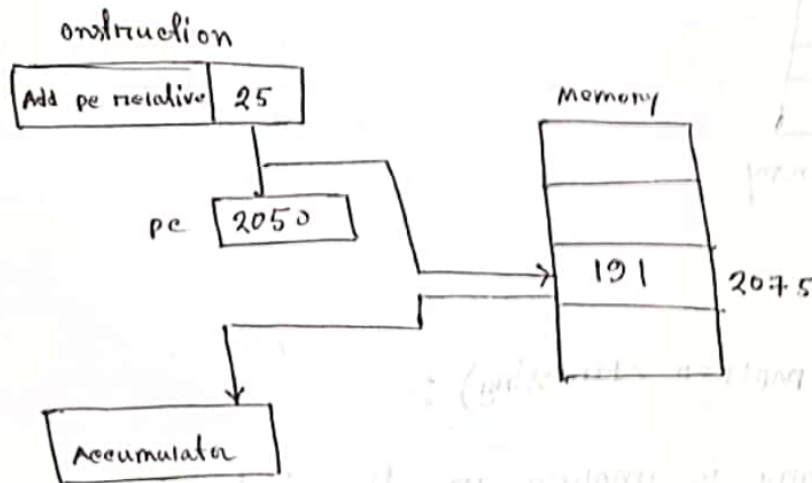
11.7 Briefly define relative addressing :

In which addressing mode operand address are found relative to program counter (PC). It is known as the relative address mode.

Algo $\Rightarrow EA = \text{Displacement} + \text{PC register}$

Example :-

Fig :-



11.8 What is the advantage of auto indexing?

In computer organization and architecture, autoindexing refers to the ability of an instruction to automatically increment or decrement the address stored in an index register. It can be useful for several reasons:

1. It simplifies the process of iterating through an array or other data structure.
2. It reduces the size of the program.
3. Improves the performance of the program by reducing the number of instruction execution.

11.9 What is the difference between postindexing and preindexing?

In computer architecture, indexing refers to the process of using an index to access elements of an array. There are two main types of indexing: preindexing and postindexing.

Preindexing :- Preindexing involves calculating the memory address of the element being accessed before accessing the element.

This is done by using an Address Calculation Unit (ACU) or an Address Generation Unit (AGU).

post indexing :- post indexing involves, accessing the element at a memory location and then using the index to calculate the memory address of the next element. This is typically done by Data path unit (DPU) or a Load store unit (LSU)

on general preindexing is more efficient than postindexing. But postindexing is more flexible than preindexing.

11.10 what facts go into determining the use of the addressing bits of an instruction?

⇒

① Number of addressing modes :-

- ② Number of operands
- ③ Register versus memory
- ④ Number of register set
- ⑤ Address range
- ⑥ Address granularity

1.11 Advantages and Disadvantages of using a variable length instruction format?

→ Variable length instruction is a type of instruction in which the size of an instruction can vary.

Advantages

- ① Reduced instruction fetch overhead
- ② More efficient encoding of instructions
- ③ Flexibility in instruction design

Disadvantage:

- ① Increased complexity
- ② Decreased performance
- ③ Increased code size



problem

(2) Justify the assertion that a 32 bit instruction is probably much less than twice as useful as a 16 bit instruction.

⇒ In general the usefulness of an instruction is determined by the capabilities it provides and how well it fits into the overall instruction set and architecture of the processor.

→ 32 bit instruction has higher overhead for instruction decoding and execution which could potentially decrease its overall usefulness compared to a smaller instruction.

That's why generally 32 bit instruction is not twice as useful as 16 bit instruction.

40	20	Address/word
50	30	"
60	40	"
70	50	"

11.2

a. Immediate 20 ⇒ 20

b. Direct 20 → 40

c. Indirect 20 ⇒ 60

Immediate 30 → 30

e. Direct 30 ⇒ 30

f: 70

11.3

3.11

$$\text{physical address} = \text{register address} + (\text{offset})$$

$$1159 = \text{R14 address} + \text{register address}$$

$$1159 - 1000 = 159$$

$$10 - 1$$

11.4 An instruction contain decimal value 14. where is the correspond located here:

- (a) immediate addressing $\rightarrow$ the address field
- (b) Direct addressing $\rightarrow$ memory location 14
- (c) Indirect addressing $\rightarrow$ The memory location whose address is in memory location 14.
- (d) Register addressing $\rightarrow$ register 14
- (e) Register indirect addressing $\rightarrow$ The memory location whose address is in register 14.

11.5

$$(PC + 1) + \text{Relative address} = \text{Effective address}$$

$$\begin{aligned}\therefore \text{Relative address} &= \text{Effective add} - (PC + 1) \\ &= 530 - 621 \\ &= -91\end{aligned}$$